

Interaktiver Modus

Commandlet-Aufbau

Verb-Noun	Verb: Get, Add, Copy, Set, ... Noun: Process, Item, Help, ...
-----------	--

Hilfefunktionen

Update-Help -UICulture en-US	Als Admin: aktualisiert Hilfedateien
Get-Help <cmd>	Zeigt Hilfe zu Cmdlet
Get-Help *Item	Zeigt alle Cmdlets, die mit Item enden
Get-Command <cmd>	Hilfe zu Commands (-Verb Get holt alle Get-Cmdlets)
Get-Help about_*	Zeigt alle About-Docs (z. B. about_if).
<var> Get-Member	Variablen und Commandlets untersuchen
<Cmd> -WhatIf	Ausführung emulieren

Aliase

New-Alias	Erstellt neuen Alias
Remove-Alias	Löscht einen Alias
Get-Alias -Definition <cmd>	Zeigt alle Aliase zu einem Commandlet

Standardkanäle

<a> i> <file>	Write Stream <i>i</i> von <i>a</i> in <i>file</i>			
<a> i>> <file>	Append Stream <i>i</i> von <i>a</i> in <i>file</i>			
<a> i>&j	Leitet Stream <i>i</i> von <i>a</i> in Str. <i>j</i>			
Nr	PowerShell	Linux	Nr	PowerShell
0		stdin	4	verbose
1	sucess	stdout	5	debug
2	error	stderr	6	information
3	warning		*	all streams

Dateisystem

Set-Location	In Pfad wechseln
Get-Location	Aktuellen Pfad holen
New-Item	Erstellt Datei oder Ordner
Copy-Item	Kopiert Datei oder Ordner
Move-Item	Bewegt Datei oder Ordner
Remove-Item	Löscht Datei oder Ordner
Get-Item	Holt Meta-Informationen eines Items (z. B. Datei) ein
Set-Item	Setzt Meta-Informationen
Get-Content	Liest Inhalt einer Datei ein
Set-Content	Ersetzt den Inhalt einer Datei
Add-Content	Fügt Inhalt in einer Datei an
Get-ChildItem	Holt ein Item und seine Kinder-Items (Unterordner)
Tree	Zeigt Ordner rekursiv in Baumstruktur (nicht PS)

Ausgaben (kleine Auswahl)

Write-Output	Erzeugt eine Ausgabe auf stdout
Write-Debug 'msg' -Debug	Schreibt eine Debugnachricht und aktiviert den Debug-Modus
Get-Help	Hilfe zu \$DebugPrefrence etc. about_PreferenceVariables

Grundlagen Vergleichsoperatoren

\$a -eq 2	Vergleich auf Gleichheit
\$a -gt 2	Vergleich auf Größer
\$a -like "W*"	Vergleich auf Wildcard-Pattern eines Strings
\$a -in \$x	Vergleich, ob in Liste enthalten

Pipelining

<a> 	Leitet stdout von <i>a</i> in stdin von <i>b</i>
Where-Object{ }	Filtert Objekte basierend auf einer Bedingung.
Select-Object	Wählt bestimmte Eigenschaften eines Objekts aus.
Sort-Object	Sortiert Objekte nach einem bestimmten Kriterium.
Foreach-Object{ }	Anweisungen pro Objekt ausführen
Group-Object	Gruppert anhand einer Eigenschaft der Objekte.
Get-Member	Metadaten zu Objekt ausgeben
Measure-Object	Min, Max, Sum, Avg
Compare-Object	Zwei Objektmengen vergleichen
Format-List	Ausgabe formatieren (viele Format-Varianten)
Tee-Object <a> 	Splittet stdout in <i>a</i> und <i>b</i> auf
Get-Help about_Pipelines	Hilfeartikel zu Pipelines
Out-Null	Unterdrückt Output in einer Pipeline
Out-Printer	Sendet Output an Drucker
Out-File	Sendet Output an Datei

Skripting

Parameter in Skripten

<code>./script.ps1 <arg1> [, ..., <argN>]</code>	
<code>\$args.Count</code>	Anzahl der Argumente prüfen
<code>\$args[i]</code>	Positionale Argumente an Stelle <i>i</i> auslesen
<code>./script.ps1 -par1 <w> [, ..., -parN <w>]</code>	
<code>param([typ]\$par1, ... [typ]\$parN)</code>	Benannte Parameter mit Typ definieren.

Umgang mit Variablen

<code>[int] \$x = 5</code>	Zuweisung einer typisierten Variablen (Typ ist optional)
<code>[int] \$x = "3.45"</code> <code>-as [Int]</code>	Konvertiert einen String-Wert in Int und schreibt ihn nach <code>\$x</code>
<code>\$x.GetType()</code>	Liefert Typinfos von <code>\$x</code>
<code>\$x.GetType().FullName</code>	Liefert Typnamen von <code>\$x</code>
<code>Clear-Variable x</code>	Löscht Inhalt von <code>\$x</code>
<code>Remove-Variable x</code>	Löscht Deklaration von <code>\$x</code>
<code>\$true \$false</code>	Wahr/falsch
<code>\$Home</code>	Home-Folder des Nutzers
<code>\$PSHome</code>	Installationsordner von PS
<code>\$Error</code>	Liste aller Fehler seit Start der PowerShell
<code>Get-Item Variable:H*</code>	Zeigt alle definierten Variablen an, die mit H beginnen
<code>\$x Get-Member</code>	Zeigt Typ, Member, Methoden zu der Variablen an
<code>Get-Help about_Variables</code>	Hilfeartikel zu Variablen

Umgang mit Strings

<code>"Hi"</code> bzw. <code>'Hi'</code> bzw. <code>@'Hi@'</code>	Versch. Stringliterationale (@-Notation: "Here-String")
<code>"a" + \$x + "c"</code>	Konkatenation
<code>"PC \$nr"</code>	Ausdruckauflösung
<code>"Date: \$(Get-Date)"</code>	Ausdruckauflösung
<code>"x:\\$((\$pc)_VHD.vhdx"</code>	Ausdruckauflösung
<code>\$a.Substring(4,3)</code>	Text extrahieren [5,7]
<code>\$myArr = \$x -Split ""</code>	Splitten String am Delimiter <code></code> auf
<code>\$x = \$myArr -Join ""</code>	Verbindet Teilstringe aus <code>myArr</code> in <code>x</code>
<code>\$x.replace("ü", "ue")</code>	Case-Sensitives Ersetzen von Teilstrings
<code>\$x -replace "\d ", "X"</code>	Ersetzen von Teilstrings m.H. von regulären Ausdrücken
<code>"" Get-Member -MemberType Method</code>	String-Methoden ansehen

Umgang mit nicht definierten Variablen

<code>\$x ??= "n/a"</code>	Nimm <code>x</code> falls definiert, ansonsten schreibe Standardwert hinein
<code>\${x}?.Property</code>	Wähle Property aus, falls existent, ansonsten <code>\$null</code> zurückgeben
<code>\${arr}[100]</code>	Falls <code>arr[100]</code> nicht existiert, gib <code>\$null</code> zurück

Ein- und Ausgabe

Write-Output	Erzeugt Ausgabe auf <code>stdout</code>
<code>\$x = Read-Host "x eingeben"</code>	Benutzereingabe wird nach <code>\$x</code> gespeichert
Clear-Host	Löscht die Ausgabe auf der Konsole

Dokumente lesen und schreiben

Get-Content	Textdatei einlesen
<code>\$x[0]</code>	Textzeile 0 auswählen
Set-Content	Textdatei überschreiben
Add-Content	Text in Textdatei anhängen
Import-Csv	Text in Textdatei anhängen
<code>\$x[0].SpaltenName</code>	Spalte in Objekt 0 auswählen
ConvertFrom-Csv	Aus String CSV extrahieren
Export-Csv	CSV-Datei schreiben
ConvertFrom-Json	Aus String JSON extrahieren
<code>\$x.propertyName</code>	<code>propertyName</code> auswählen
ConvertTo-Json	JSON-String erzeugen

Arrays

<code>\$x = "a","b","c"</code>	Array definieren
<code>\$x = @(1,2,3)</code>	Array definieren
<code>\$x = 1..10</code>	Zahlen 1 bis 10 in x schreiben
<code>\$x.Length</code>	Anzahl der Elemente holen
<code>\$z = \$x + \$y</code>	Zwei Arrays verbinden
<code>\$x[-3..-1]</code>	Letzte 3 Elemente holen
<code>\$x[1,4+6..9]</code>	Elemente 1,4,6 bis 9 holen
<code>\$x = ("a", "b"), ("c", "d")</code>	Zwei-dimensionales Array erzeugen
<code>\$x[0][1]</code>	Element "b" an (0,1) holen
<code>\$x = @{"a" = "w1"; b = "w2"}</code>	Assoziatives Array erzeugen
<code>\$x["a"]</code> bzw. <code>\$x.a</code>	Wert von Index "a" auslesen
<code>\$Get-Help about_Arrays</code>	Hilfeartikel zu Arrays aufrufen

Schleifen

<code>for (\$i=1; \$i -lt 6; \$i++) { anweisungen() }</code>	zählergesteuert
<code>while(\$i -lt 5) { }</code>	kopfgesteuert
<code>do { } while(\$i -lt 5)</code>	fußgesteuert
<code>do { } until(\$i -eq 5)</code>	mit Abbruchbed.
<code>foreach (\$i in \$menge) { }</code>	elementgesteuert

Verzweigungen

<code>if (condition){ } elseif{ } else{ }</code>	If-Abfrage
<code>Condition ? True : False</code>	Ternärer Operator
<code>switch(\$x) { 1 { \$y= "A" } 2 { \$y= "B" } }</code>	Switch-Case Abfrage: x=1 ? dann y = "A" usw.

Funktionen

Alles was nicht `Write-*` ausgegeben wird, wird zum **Rückgabewert** der Funktion

<code>function Get-Alter()</code>	positionale Parameter
<code>function Get-Alter(\$age)</code>	benannte Parameter
<code>function Get-Age([int]\$age)</code>	benannte und typisierte Parameter
<code>function Get-Alter() { param([int] \$age) }</code>	Variante 2
<code>Get-Alter 20</code>	Aufruf mit positionalem Parameter
<code>Get-Alter -age 30</code>	Aufruf mit benanntem Parameter
<code>return</code>	Funktion (vorzeitig) verlassen.

Fehlerbehandlung

<code>\$?</code>	<code>\$True</code> für Erfolg, <code>\$False</code> für Fehlschlag
<code><a> && </code>	Führt <code></code> aus, falls <code><a></code> erfolgreich
<code><a> </code>	Führt <code></code> aus, falls <code><a></code> nicht erfolgreich
<code>try{ #unsafe } catch{ #catchIt } finally{ #after }</code>	Try unsafe code, Catch error and finally do something
<code>\$_</code>	Zugriff auf Error-Objekt innerhalb des catch-Blocks

<code>Write-Error "Fehler bei ... \$_"</code>	Fehlerrückmeldung im Catch-Block mit Zugriff auf Error-Objekt
<code>throw "Fehler: Skript gestoppt"</code>	Einen eigenen Fehler erzeugen
Error-Action und Warning-Action: Stop, Continue, SilentlyContinue, Inquire	
<code>Get-Help about_CommonParameters</code>	

Reguläre Ausdrücke

<code>\$z -match "Peter"</code>	Liefert \$true bzw. \$false zurück. Falls \$true steht Ergebnis in \$matches[0]
<code>\$z -match "^Peter"</code>	Anker Textbeginn
<code>\$z -match "Peter\$"</code>	Anker Textende
<code>\$z -match "\w"</code>	Prüfe auf word: alphanumerische Zeichen
<code>\$z -match "\d"</code>	Digit
<code>\$z -match "\s"</code>	Whitespace
<code>\$z -match "\D "</code>	Not Digit (geht bei allen Character Classes)
<code>\$z -match "[äöüß]"</code>	Character Set: z. B. nur dt. Umlaute finden
<code>\$z -match "\d{4,} "</code>	Quantifiers: mindestens 4x hintereinander
<code>+ , * {1,3}, {2}</code>	von links nach rechts: mind. 1x, beliebig oft, 1 bis 3 Mal, genau 2 Mal
<code>\$z = "alf@beta.com"</code>	Capturing Groups:
<code>"\$z -match "(\\w*)(@)(\\w*)\\.(com)"</code>	Jeder Ausdruck in Klammern kann über <code>\$matches[i]</code> abgerufen werden. <i>i</i> steht für die Klammer (von 0 beginnend)